# Sentiment Classification Interpretability on IMDb Movie Reviews Applying Shapley Additive Explanations (SHAP)

Eileen Murphy

Student Number: 501113406

Supervisor: Ashok Bhowmick, Ph.D

Date of Submission: October 30, 2026

**Abstract**

In this paper, we propose using the IMDb Movie Reviews dataset to conduct sentiment analysis and apply SHAP to understand the words or phrases that contribute to its score. Taking the sentiment analysis out of the "black box" to having transparent scores gives users the ability to validate and correct while improving the model over time. In this way, the model will become an asset rather than a potential unknown liability. The problem with artificial intelligence is not the effectiveness, but the trust of people using it. Models are not perfect and sometimes models will fail in just one reading, but that could be the only reading people see and dismiss the model entirely, as a result.

Using Krishnakumar & Abdou (2020) as a theme, research on arrhythmias with different diagnosed classifications provides an insightful look on how different patterns in the range of their measurements can not only enhance their diagnosis but also make it transparent on which measurements were the most indicative of their condition. Another important aspect is the "bellwether" methodology to filter high quality instances to train the model to improve its performance scores.

## Proposed Approach

This research will train four classification models using Python SKLearn and/or KERAS:

1. Logistic Regression (TF-IDF)
2. Naive Bayes (TF-IDF)
3. Random Forest (TF-IDF)
4. LSTM (RNN) with Word Embeddings
5. DistilBERT Transformer (? Maybe)

SHAP values will be computed using Python SHAP packages on the models and compared to each other. Different plots will be generated to clarify how tokens affect the predictions.

"Bellwether" instances will be identified from partitioning the data (ie SHAP scores, movie genre, or other possibilities). Once the dataset is partitioned then we will run tests to see if a bellwether partition exists. If there is a valid bellwether partition, then the model will be retrained on the bellwether subset. Finally, evaluation metrics will be calculated to assess model performance.

## Research Questions

RQ1: How do SHAP explanations differ across traditional ML, deep learning, and transformer models for sentiment classification?

RQ2: Does SHAP-based "bellwether" instance selection improve model performance?

RQ3: Which sentiment-driving words are consensus across models, and which are model-specific?

## Introduction

The IMDB dataset has been used as a benchmark dataset for NLP sentiment analysis since 2011, and papers have been published by raising the accuracy of the previous study. As of this writing, it has reached 95-97% accuracy by 2019 (Ruder, 2019). A link to the leaderboard of these scores and its respective authors' papers are a treasure trove of the historical evolving methods of achieving the highest accuracy as possible and the path getting us there.

Approaches used on NLP sentiment analysis used in studies reveal that attaining the highest accuracy is not linear and there are some tradeoffs between accuracy, robustness and computational cost.

Testing and measuring the robustness of the model, in some cases tweaking the dataset to see how the model responds was not possible for "black box" models since their scoring was not transparent. Meanwhile in 2017, a paper was published establishing a framework called SHAP(SHapley Additive exPlanations) (Lundberg, 2018). By 2018, Python came out with the first version of the SHAP framework package. Being able to access the SHAP framework using Python made it possible for the higher performing "black box" models to be studied for their robustness in addition to their accuracy. Measuring robustness is not standardized, but using various techniques of comparison has shown that the new models are not just more accurate but more robust but only at the cost of increased computational processing. So now the challenge is finding the balance between accuracy, robustness, and computational processing.

## Dataset

The IMDb Movies dataset will be used in this paper since it has been a reliable source for benchmarking for sentiment classification since 2011 and our results can be directly compared to other studies that still use it today.

According to its documentation (Maas et al., 2011), the database contains 50K records split evenly between 25K for training and 25K for testing.

Each record includes the raw text of the review, rating (1-10), sentiment (pos or neg). The sentiment is derived from the ratings. Ratings <= 4 are labeled negative, and ratings >= 7 are rated positive. There are no neutral ratings (5-6) included in the training or test datasets.

In the IMDB dataset there is another 50K records that include all the ratings (1-10). It does not include the derived sentiment from the ratings used in the labelled test and training sets. The

purpose of including this dataset is for unsupervised learning. Although, we may use this dataset for our own purposes in this study.

IMDb Movies Database Source (Maas et al., 2011)

## Related Work

In the "Bellwethers: A Baseline Method For Transfer Learning" paper, published on January 2018 (Krishna & Menzies, 2018) is a detailed and rigorous paper of establishing a bellwether subset of data and using transfer learning on the other partitioned subsets of data. This paper was written to solve the problem of software analytics prediction models coming to different conclusions as the number of projects increased. The author's study includes a rigorous identification of a bellwether subset, and making sure that the bellwether is tested periodically to make sure that it's still valid

The author points out that finding a bellwether subset of data is more effective in a performance boost than by using more complex models on the whole dataset.

One useful metric that can be used in this paper is the confusion square metric that calculates the result in a single value. The formula is as follows:

The paper defines G-Score as:

$$G = \frac{2 \times P_d \times (1 - P_f)}{1 + P_d - P_f}$$

Where:

- $P_d$ = Probability of Detection = $\frac{TP}{TP + FN}$ (Recall / True Positive Rate)

- $P_f$ = Probability of False Alarm = $\frac{FP}{FP+TN}$ (False Positive Rate) (Krishna & Menzies, 2018)

"Explainable Sentiment Analysis: A Comparative Study of BERT, DistilBERT, and TF–IDF on the IMDB Dataset Using LIME and SHAP" (Khan et al., 2026) compares traditional machine learning models and transformer based models using their accuracy scores. In addition, the models use LIME and SHAP model agnostic explanation frameworks to find token level importance, explainability, and robustness.

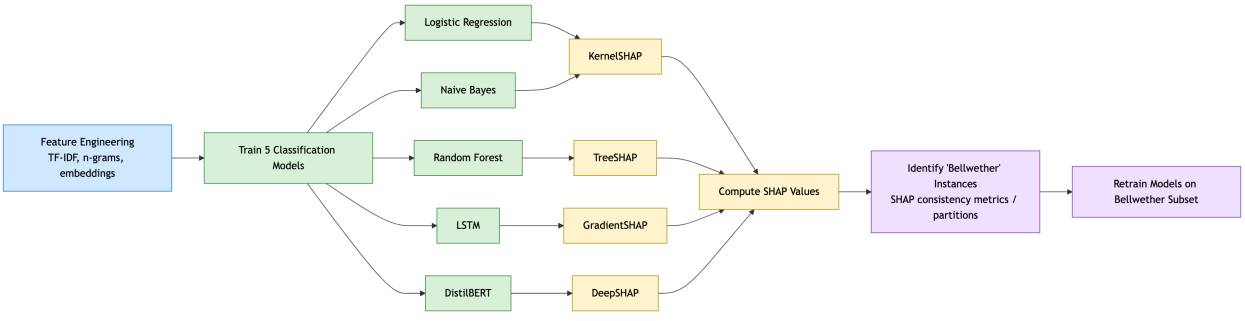
To test robustness, it took out the top important tokens and noted the accuracy difference. Some models like SVM were dependent on some words for their accuracy scores while BERT maintained its accuracy even with 10% of the most important terms were dropped. The paper was concerned by not just performance scores but by whether the model was based on keyword tokens for their scores or if the model had a deeper understanding of the token taken in context. In its conclusion, DistilBERT model provided the best tradeoff between computation overhead, robustness, and accuracy scores.

A comparative study of using different machine learning and ensemble methods titled, "Binary Sentiment Classification on the IMDB Movie Review Dataset Using An Ensemble (Random Forest, Logistic Regression, and Xgboost) Approach." (Dongre et al., 2025) In this paper the authors compared using Logistic Regression, Random Forest, XGBoost, and Ensemble with detailed preprocessing and analysis for accuracy and F1 results and produced a confusion square for each test. The authors concluded that the Ensemble (Soft Voting) had the highest accuracy but only slightly above the Logistic Regression model, but was more stable.

The value of the "Advanced Sentiment Analysis of IMDB Movie Reviews Using NLP Preprocessing, TF-IDF and Classical Machine Learning Models" (Limbola et al., 2026) is its detail on pre-processing and EDA analysis of the IMDB dataset and the description of the inner workings each machine model being examined. The authors compared Machine Learning models (Logistic Regression, SVM, Naive Bayes, Random Forest) with deep learning models (LSTM and BiLSTM). The test results showed that the Logistic Regression model is still a competitive model in its effectiveness, interpretation, and computational efficiency to be used as a baseline despite the advances in deep learning models.

"Sentiment Analysis on IMDB Reviews: A Comparative Study of Bag-of-Words, Deep Learning, and Transformer Models," (Shetty et al., 2026) is another comparison study but includes transformer architecture and compares Bag of Words (BOW), TF-IDF, and deep learning GloVe embeddings and pre-trained transformers. The conclusion was that the transformer model (Longformer) surpassed the other models (BERT and DistilBERT), and there is high potential to improve them.

## Methodology




```
train shape:  (25000, 4) test shape:  (25000, 4)

Train columns:  ['text_raw', 'text_clean', 'label', 'rating']
```

## Taking the 25k Training dataset and splitting it 80/20

```
X_train: (20000,)  X_test: (5000,)

y_train: (20000,)  y_test: (5000,)

train class balance:

 label

1    0.5

0    0.5

Name: proportion, dtype: float64

test class balance:

 label

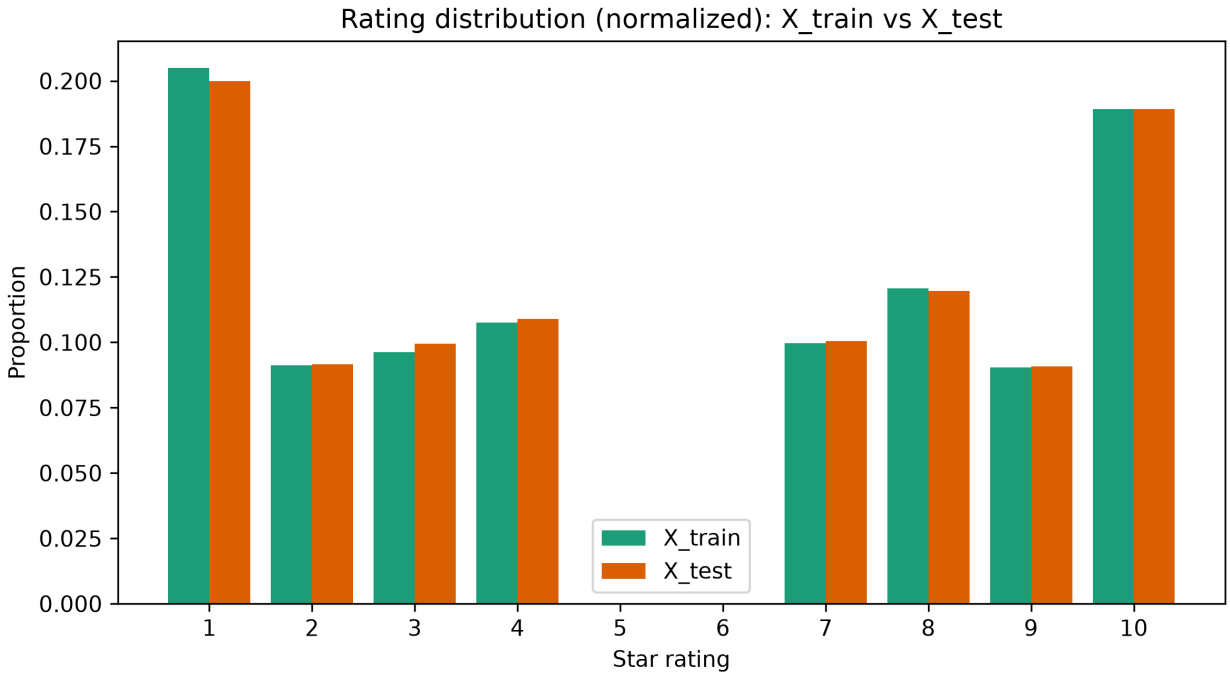
1    0.5

0    0.5

Name: proportion, dtype: float64
```

## Rating distribution

The distribution of the ratings is U shaped, while the sentiment (pos/neg) is evenly distributed




The proportions line up closely across every rating value, confirming the stratified split preserved the original U-shaped rating distribution (ratings of 1 and 10 dominate, with no 5-6 ratings present).

## Review length distribution by label

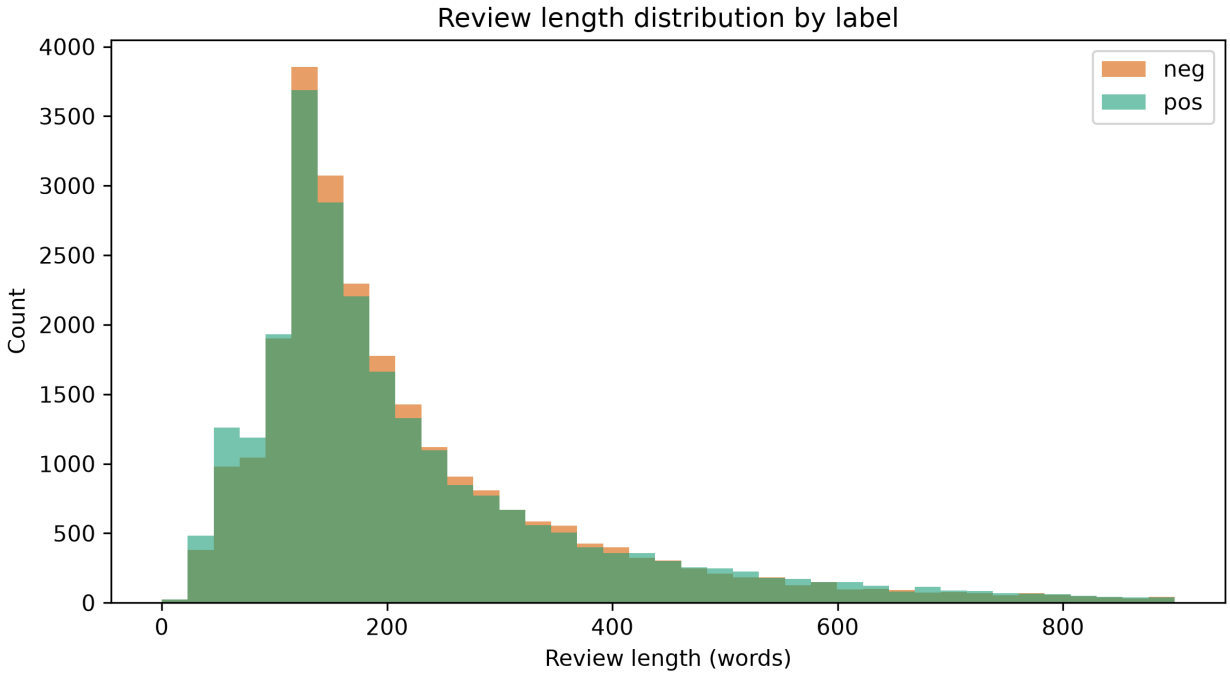



Figure 1

The two distributions overlap almost completely and are both strongly right-skewed, confirming that review length does not meaningfully distinguish positive from negative reviews on average.

| | label | mean | median | max | n |
|---|---|---|---|---|---|
| 0 | 0 | 227.87484 | 173.0 | 1509 | 25000 |
| 1 | 1 | 231.51840 | 171.0 | 2460 | 25000 |

## Review length distribution by split

| | split | mean | median | max | n |
|---|---|---|---|---|---|
| 0 | test | 227.07664 | 171.0 | 2240 | 25000 |
| 1 | train | 232.31660 | 173.0 | 2460 | 25000 |

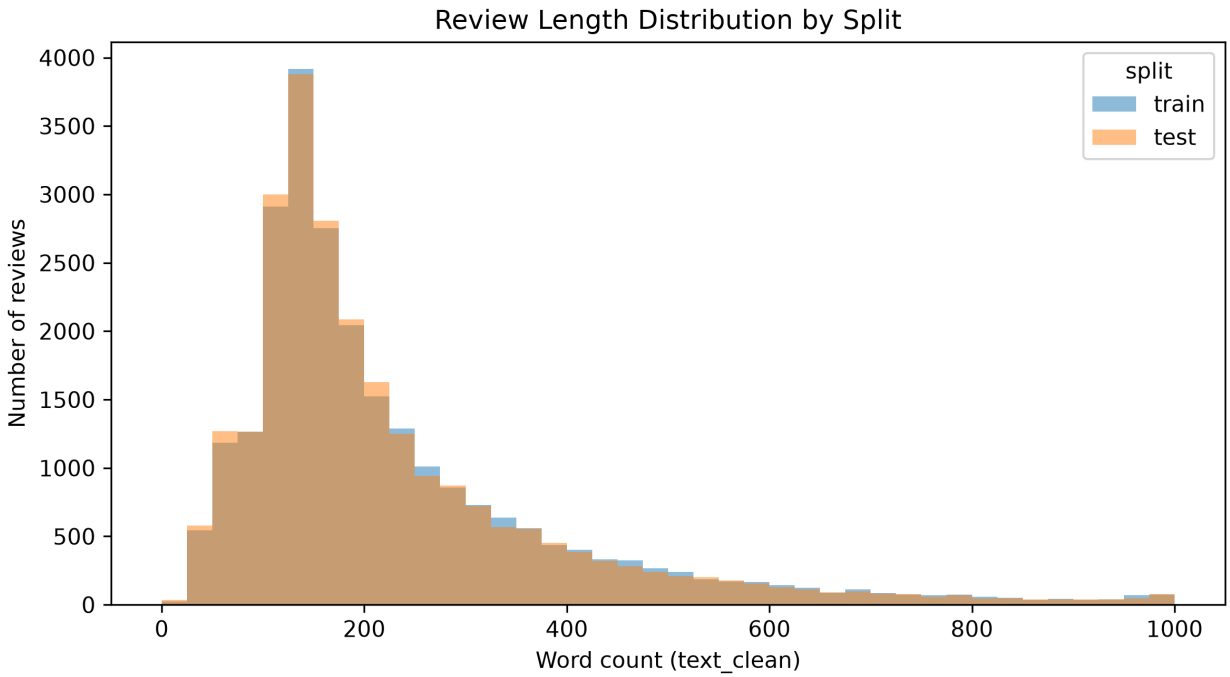



Figure 2

Review length distributions are also very similar between the `train` and `test` splits, with closely matching means, medians, and overall shape. This indicates the train/test split does not introduce a length-based bias.

## Feature Extraction

### Unigrams using CountVectorizer

```
Unigram counter matrix shape: (20000, 70538)
```

```
Unigram features using counter

Shape: 20000 x 70538

Non-zero entries: 1,787,417

Density: 0.001267 (0.1267%)

Sparsity: 0.998733 (99.8733%)

Avg non-zero terms per document: 89.4

      term  freq_count

0    movie       35227

1     film       31240

2     like       16396

3     just       14247

4     good       12167

5     time       10204

6    story        9507

7   really        9413

8      bad        7411

9   people        7373

10   great        7247

11    dont        6877

12  movies        6563

13     way        6462

14    make        6438
```

**Unigrams using TF-IDF**

```
Unigram tfid matrix shape: (20000, 70538)

Unigram features using Tfid

Shape: 20000 x 70538

Non-zero entries: 1,787,417

Density: 0.001267 (0.1267%)

Sparsity: 0.998733 (99.8733%)

Avg non-zero terms per document: 89.4

      term   freq_tfid

0    movie  961.947329

1     film  776.245237

2     like  452.172358

3     just  427.665118

4     good  406.913264

5   really  342.762510

6    story  336.876896

7     time  334.326304

8      bad  322.648217

9    great  295.571260

10  movies  288.114173

11    dont  281.385490

12  people  280.529406
```

```
13   watch  258.280574

14   think  250.224006
```

**BiGrams using TF-IDF**

```
BiGram tfid matrix shape: (20000, 206203)

Ngrams features using Tfid

Shape: 20000 x 206203

Non-zero entries: 905,929

Density: 0.000220 (0.0220%)

Sparsity: 0.999780 (99.9780%)

Avg non-zero terms per document: 45.3

               term  freq_tfidf_2

0          ive seen     89.966367

1   special effects     60.650624

2         dont know     60.403217

3        waste time     55.315566

4        low budget     50.360572

5          year old     49.580197

6        good movie     48.616912

7        looks like     47.971454

8       watch movie     46.294976

9        movie just     45.876615

10          im sure     42.891206
```

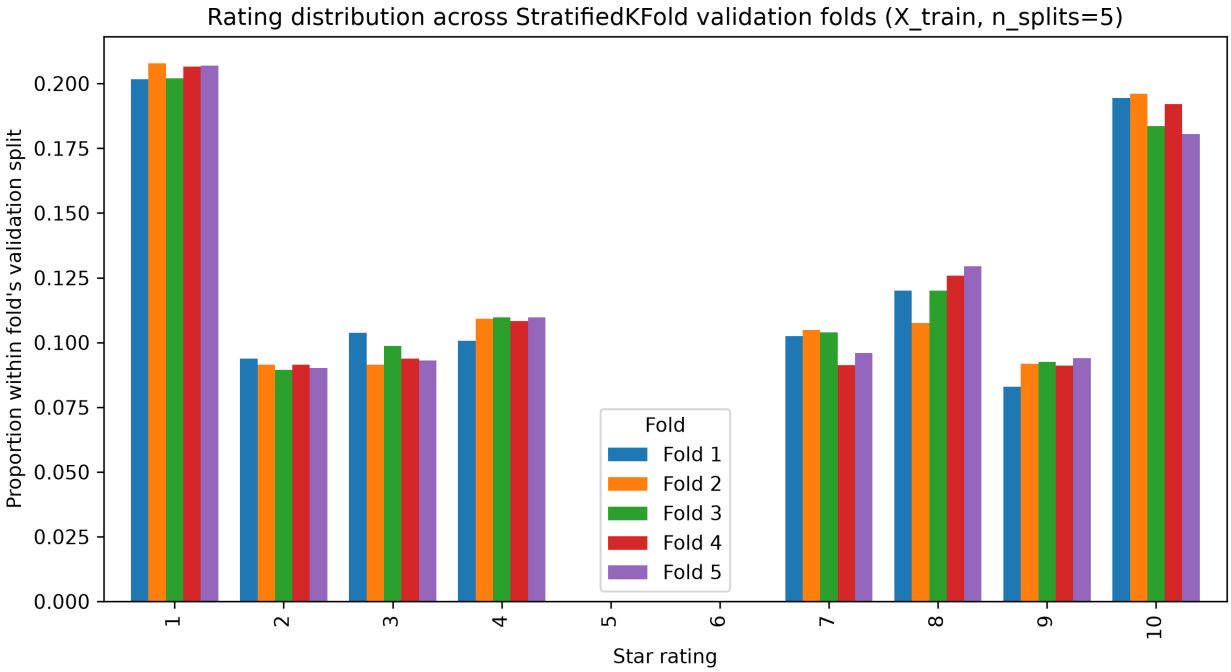
| | | |
|---|---|---|
| 11 | new york | 41.855528 |
| 12 | dont think | 41.317676 |
| 13 | years ago | 39.575503 |
| 14 | sci fi | 38.893582 |

## Cross-Validation on X_train

Cross-validation is run on `X_train`/`y_train` only (not the full 25k or `X_test`), so that `X_test` remains an untouched development hold-out and `imdb_test.csv` remains available later as the final blind test set. Settings (TF-IDF `max_features=10000`, `min_df=2`, `max_df=0.95`; 5-fold `StratifiedKFold`) match the project's existing baseline (`scripts/cv_model_comparison.py`) for comparability.

### Rating distribution across CV folds

`StratifiedKFold` stratifies on the binary `label`, not on `rating` directly, so it's worth checking that each fold's validation split still preserves a similar U-shaped rating distribution to the overall `X_train` set.




## Initial Results

```
========================================================================

VECTORIZER: TF-IDF

========================================================================


Logistic Regression  (elapsed 65.7s)

  accuracy   test: 0.8859 (+/- 0.0026)   train: 0.9269

  f1         test: 0.8871 (+/- 0.0022)   train: 0.9274

  roc_auc    test: 0.9539 (+/- 0.0023)   train: 0.9791


Naive Bayes  (elapsed 66.1s)
```

```
  accuracy   test: 0.8562 (+/- 0.0048)   train: 0.8878

  f1         test: 0.8544 (+/- 0.0046)   train: 0.8864

  roc_auc    test: 0.9322 (+/- 0.0027)   train: 0.9565


Random Forest  (elapsed 167.2s)

  accuracy   test: 0.8411 (+/- 0.0043)   train: 0.9707

  f1         test: 0.8425 (+/- 0.0032)   train: 0.9710

  roc_auc    test: 0.9197 (+/- 0.0032)   train: 0.9964


======================================================================

VECTORIZER: CountVectorizer

======================================================================


Logistic Regression  (elapsed 36.0s)

  accuracy   test: 0.8679 (+/- 0.0044)   train: 0.9975

  f1         test: 0.8686 (+/- 0.0044)   train: 0.9975

  roc_auc    test: 0.9364 (+/- 0.0019)   train: 1.0000


Naive Bayes  (elapsed 27.4s)

  accuracy   test: 0.8395 (+/- 0.0056)   train: 0.8668

  f1         test: 0.8356 (+/- 0.0055)   train: 0.8635

  roc_auc    test: 0.9114 (+/- 0.0031)   train: 0.9360
```

```
Random Forest  (elapsed 103.3s)
  accuracy   test: 0.8491 (+/- 0.0039)   train: 0.9523
  f1         test: 0.8521 (+/- 0.0029)   train: 0.9530
  roc_auc    test: 0.9259 (+/- 0.0036)   train: 0.9909

======================================================================
VECTORIZER: bigram_vectorizer_tfidf
======================================================================

Logistic Regression  (elapsed 33.3s)
  accuracy   test: 0.8427 (+/- 0.0069)   train: 0.9837
  f1         test: 0.8475 (+/- 0.0061)   train: 0.9838
  roc_auc    test: 0.9200 (+/- 0.0041)   train: 0.9979

Naive Bayes  (elapsed 42.5s)
  accuracy   test: 0.8572 (+/- 0.0067)   train: 0.9892
  f1         test: 0.8544 (+/- 0.0066)   train: 0.9892
  roc_auc    test: 0.9347 (+/- 0.0042)   train: 0.9991

Random Forest  (elapsed 117.3s)
  accuracy   test: 0.7631 (+/- 0.0089)   train: 0.8020
  f1         test: 0.7884 (+/- 0.0060)   train: 0.8234
  roc_auc    test: 0.8610 (+/- 0.0072)   train: 0.9102
```

```
======================================================================
SAVING RESULTS...
======================================================================
  Results saved to: cv_results_comparison.csv


Results Summary:
metric                                          accuracy        f1   roc_auc
model
CountVectorizer + Logistic Regression            0.86790  0.868569  0.936442
CountVectorizer + Naive Bayes                    0.83950  0.835561  0.911374
CountVectorizer + Random Forest                  0.84905  0.852122  0.925867
TF-IDF + Logistic Regression                     0.88590  0.887148  0.953855
TF-IDF + Naive Bayes                             0.85620  0.854432  0.932165
TF-IDF + Random Forest                           0.84110  0.842459  0.919661
bigram_vectorizer_tfidf + Logistic Regression    0.84270  0.847468  0.919957
bigram_vectorizer_tfidf + Naive Bayes            0.85725  0.854368  0.934739
bigram_vectorizer_tfidf + Random Forest          0.76310  0.788387  0.861016


Cross-Validation Summary:
                                          accuracy  accuracy_std      f1  \
TF-IDF + Logistic Regression                0.8859        0.0026  0.8871
```

```
TF-IDF + Naive Bayes                              0.8562        0.0048  0.8544

TF-IDF + Random Forest                            0.8411        0.0043  0.8425

CountVectorizer + Logistic Regression             0.8679        0.0044  0.8686

CountVectorizer + Naive Bayes                     0.8395        0.0056  0.8356

CountVectorizer + Random Forest                   0.8490        0.0039  0.8521

bigram_vectorizer_tfidf + Logistic Regression     0.8427        0.0069  0.8475

bigram_vectorizer_tfidf + Naive Bayes             0.8572        0.0067  0.8544

bigram_vectorizer_tfidf + Random Forest           0.7631        0.0089  0.7884


                                                  f1_std

TF-IDF + Logistic Regression                      0.0022

TF-IDF + Naive Bayes                              0.0046

TF-IDF + Random Forest                            0.0032

CountVectorizer + Logistic Regression             0.0044

CountVectorizer + Naive Bayes                     0.0055

CountVectorizer + Random Forest                   0.0029

bigram_vectorizer_tfidf + Logistic Regression     0.0061

bigram_vectorizer_tfidf + Naive Bayes             0.0066

bigram_vectorizer_tfidf + Random Forest           0.0060


================================================

Best Vectorizer + Model: TF-IDF + Logistic Regression

  Vectorizer: TF-IDF
```

```
  Model:      Logistic Regression
CV Accuracy: 0.8859 (+/- 0.0026)
CV F1:       0.8871 (+/- 0.0022)
================================================
```

Table 3

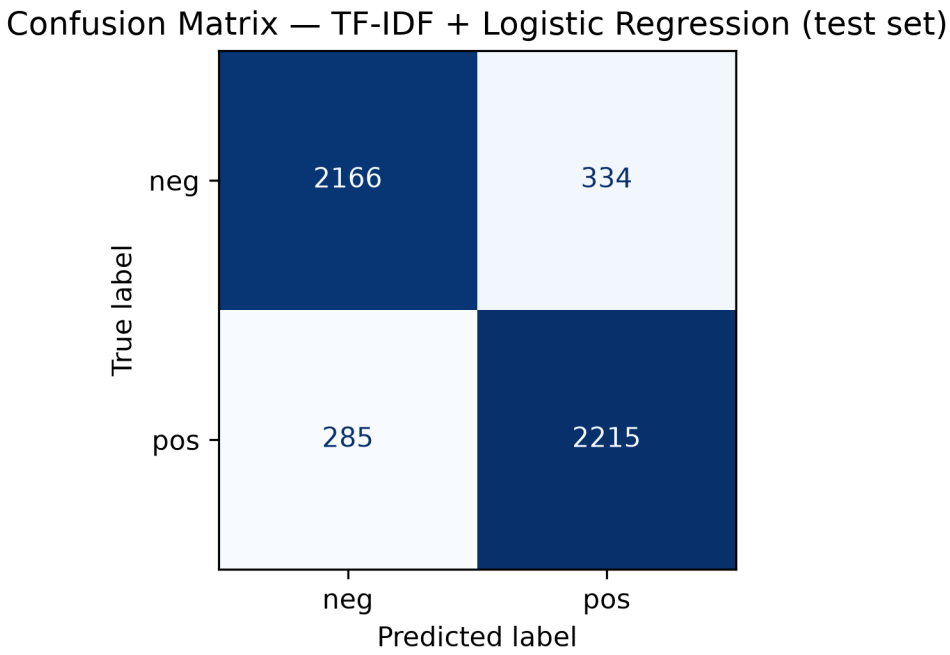
| | vectorizer | model | accuracy | accuracy_std | f1 | f1_std |
|---|---|---|---|---|---|---|
| 0 | TF-IDF | Logistic Regression | 0.8859 | 0.0026 | 0.8871 | 0.0022 |
| 1 | CountVectorizer | Logistic Regression | 0.8679 | 0.0044 | 0.8686 | 0.0044 |
| 2 | bigram_vectorizer_tfidf | Naive Bayes | 0.8572 | 0.0067 | 0.8544 | 0.0066 |
| 3 | TF-IDF | Naive Bayes | 0.8562 | 0.0048 | 0.8544 | 0.0046 |
| 4 | CountVectorizer | Random Forest | 0.8490 | 0.0039 | 0.8521 | 0.0029 |
| 5 | bigram_vectorizer_tfidf | Logistic Regression | 0.8427 | 0.0069 | 0.8475 | 0.0061 |
| 6 | TF-IDF | Random Forest | 0.8411 | 0.0043 | 0.8425 | 0.0032 |
| 7 | CountVectorizer | Naive Bayes | 0.8395 | 0.0056 | 0.8356 | 0.0055 |
| 8 | bigram_vectorizer_tfidf | Random Forest | 0.7631 | 0.0089 | 0.7884 | 0.0060 |

Table 1 (above) ranks all nine vectorizer/model combinations by cross-validated accuracy. TF-IDF + Logistic Regression achieved the highest cross-validated accuracy and F1 of all nine vectorizer/model combinations tested, consistent with the project's earlier baseline findings: Logistic Regression's linear decision boundary over sparse TF-IDF features suits this high-dimensional, near-linearly-separable bag-of-words text classification task well, while Naive Bayes' independence assumption and Random Forest's axis-aligned tree splits are comparatively less suited to

sparse TF-IDF representations, and the bigram vectorizer under-performed unigram TF-IDF, likely due to increased feature sparsity without a compensating gain in discriminative signal.

The bigram TF-IDF vectorizer underperformed unigram TF-IDF for every model except Naive Bayes (0.857 vs. 0.856 accuracy, essentially unchanged), with the largest drops for Random Forest (0.763 vs. 0.841 accuracy, a 7.8-point loss) and Logistic Regression (0.843 vs. 0.886 accuracy, a 4.3-point loss), suggesting that bigrams' much sparser, higher-dimensional feature space dilutes the signal available per feature without a corresponding gain in discriminative information.

```
Final Test Set Evaluation:
  Accuracy:  0.8762
  F1:        0.8774
  Precision: 0.8690
  Recall:    0.8860
Confusion matrix (rows = actual, cols = predicted):
                Predicted neg   Predicted pos
  Actual neg             2166            334
  Actual pos              285           2215
```




## SHAP Explanations (KernelSHAP) for the Best Model

With `best_pipe` identified above (TF-IDF + Logistic Regression), we compute SHAP (SHapley Additive exPlanations) values using `shap.KernelExplainer`, a model-agnostic explainer that treats the pipeline as a black box via its `predict_proba` method.

KernelSHAP's runtime scales with both the number of background samples and the number of instances explained, so two steps keep it tractable on a 10,000-feature TF-IDF representation:

1. **Background summarization**: rather than using the full 20,000-row `X_train` as the background distribution (which made `shap.kmeans` alone take ~14 minutes), we first randomly sample 300 training rows and then summarize those into 50 k-means cluster centers. This cuts background preparation to a few seconds with no meaningful loss in explanation quality.
2. **Explained sample size**: KernelSHAP is run on the first 50 reviews of `X_test` (with `nsamples=200` coalition samples per instance), which takes roughly a minute — enough to

see clear, interpretable patterns without the cost of explaining the full test set.

SHAP values are cached to disk (one `.npz` file per model under `shap_cache/`) after the first computation. The cache key encodes the model name, explained sample size, and `nsamples`, so later sections (or re-renders) that need these same SHAP values can load them in under a second instead of re-running the KernelSHAP computation — and the cache is automatically invalidated (recomputed) if any of those settings change.

```
[TF-IDF + Logistic Regression] Loaded cached KERNELSHAP values from shap_cache/shap_valu
SHAP values ready for 50 test reviews (10000 TF-IDF features).
```

**Global token importance**

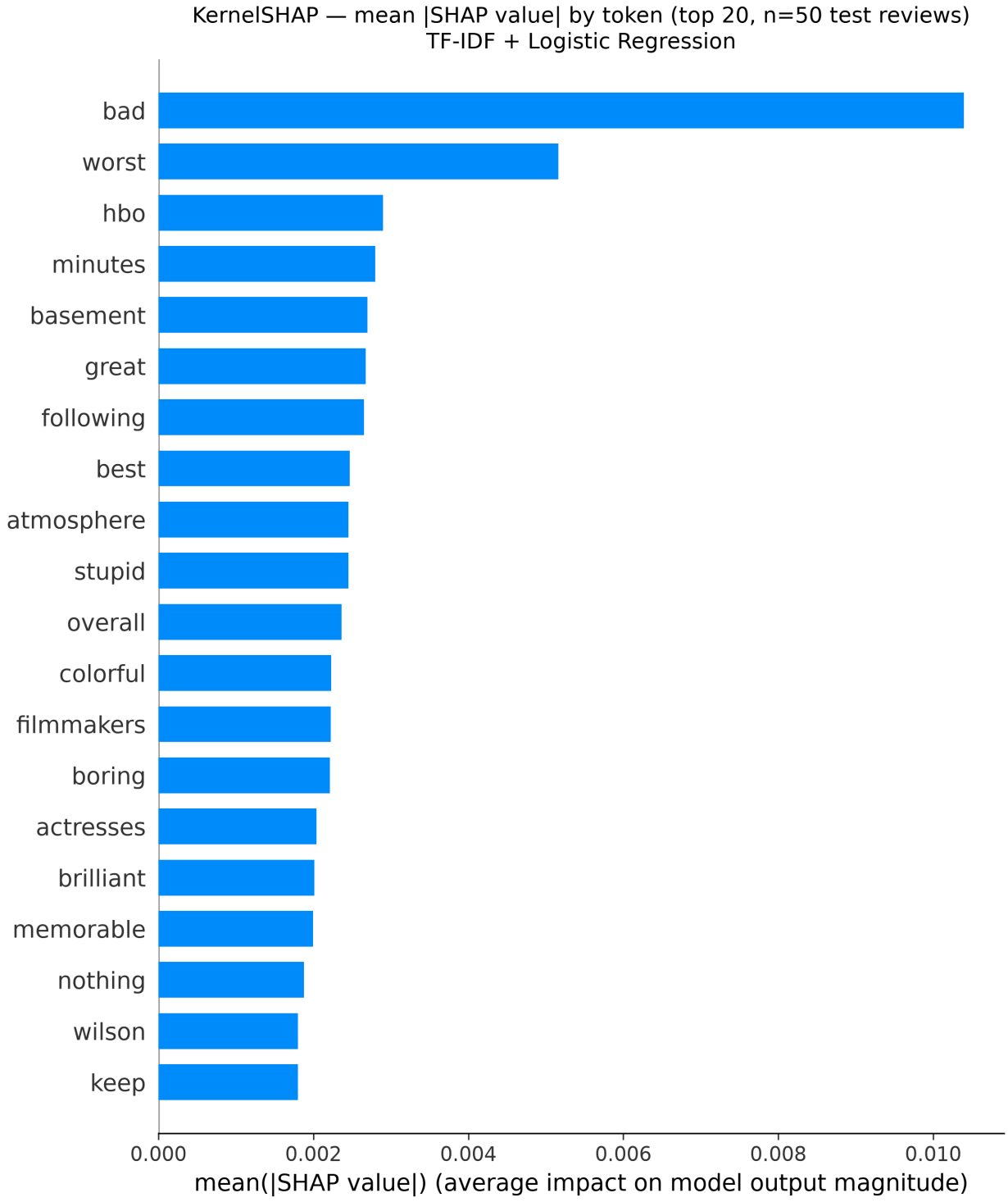



Figure 3

The top tokens by mean absolute SHAP value are intuitively sentiment-bearing (e.g. “bad”, “worst”, “stupid”, “boring” pushing toward negative; “great”, “excellent”, “love”, “best”, “amazing” pushing toward positive), confirming that the model is picking up genuine sentiment signal rather than spurious correlations.

**Single-review explanation**

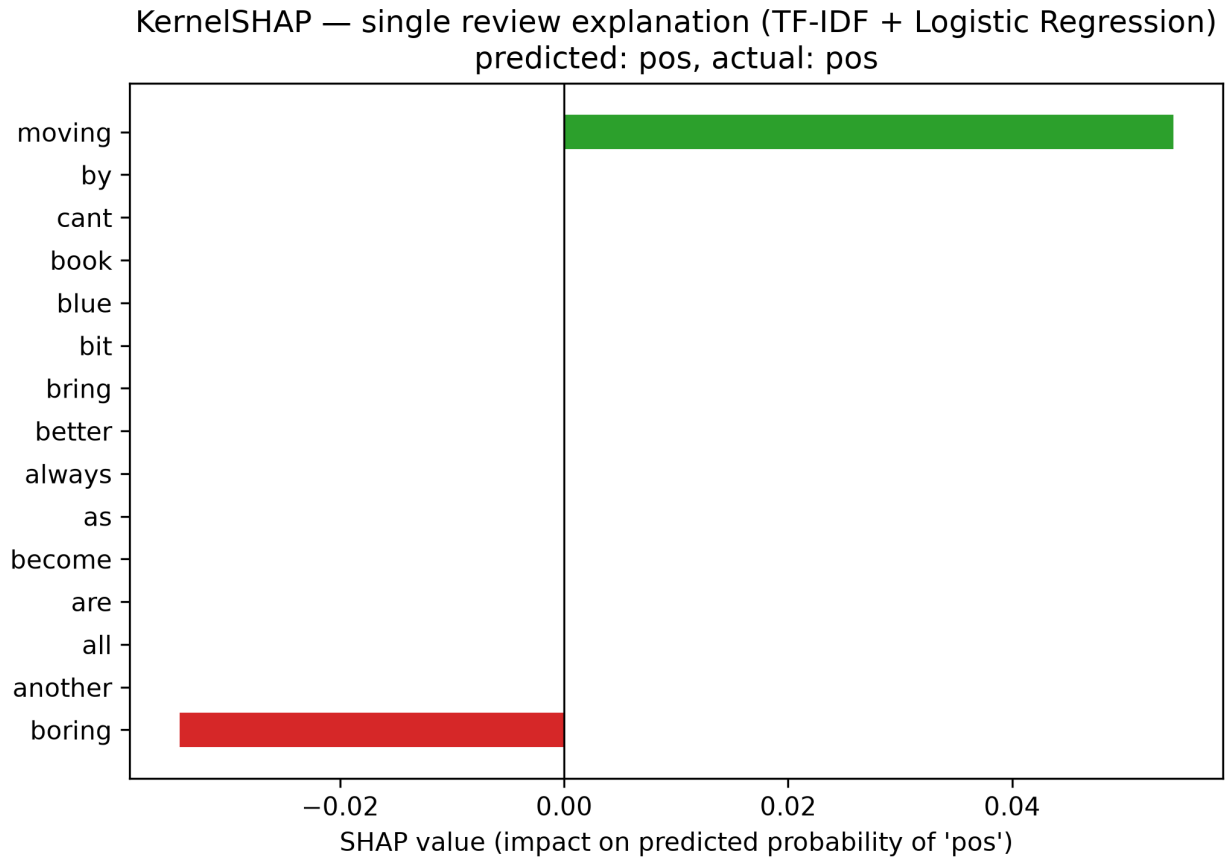



Figure 4

```
the daughters words are poetry i cant go on another year i got to get to a hotel room i
```

This review is deliberately chosen (rather than picked by sheer magnitude of SHAP values alone, which can select a review dominated by a single one-sided word and show only one bar color) to

have a clear, strong contributor pulling toward each class, so both green ("pos") and red ("neg") bars are visible — illustrating at the individual-review level the same token-level, signed reasoning seen in aggregate above.

## SHAP Explanations for Naive Bayes and Random Forest (Cross-Model Comparison)

To compare SHAP explanations across model types (RQ1/RQ3), Naive Bayes and Random Forest are each refit on the **same TF-IDF vectorizer** used for the best Logistic Regression model above, rather than each model's own best-performing vectorizer from Table 3 (bigram TF-IDF for Naive Bayes, CountVectorizer for Random Forest). Keeping the feature space identical across models is what makes token-level SHAP comparison meaningful — comparing SHAP values computed over different vectorizers would mean comparing different feature spaces (e.g. unigrams vs. bigrams), not a true cross-model comparison.

```
[TF-IDF + Naive Bayes] Loaded cached KERNELSHAP values from shap_cache/shap_values_tf-id
[TF-IDF + Random Forest] Loaded cached TREESHAP values from shap_cache/shap_values_tf-id
SHAP values ready for all three models: ['TF-IDF + Logistic Regression', 'TF-IDF + Naive
```

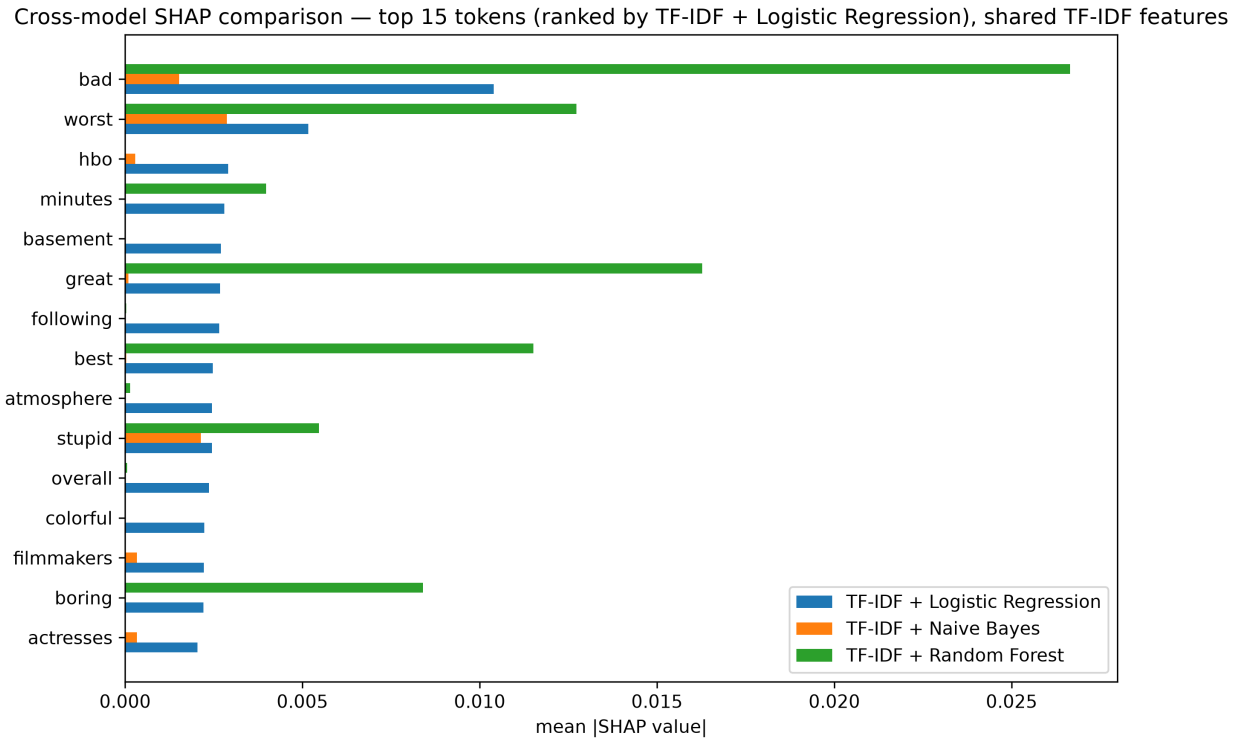



Figure 5

This early cross-model comparison, using the shared TF-IDF feature space, shows how much a token's SHAP attribution varies by model type — a starting point for the full cross-model consensus analysis (RQ3) planned later in this project.

## References

Dongre, R., Yadav, H., & Shrivastava, A. K. (2025). Binary sentiment classification on the IMDB movie review dataset using an ensemble approach. *2025 IEEE 1st International Conference on Smart Innovations in Systems, Infrastructure, Mechanical, Power, AI and Computing Technologies (SISIMPACT)*, 791–796. https://api.semanticscholar.org/CorpusID:286875533

Khan, M. M. Z., Prottoy, F. S., Akter, F., Aurpa, T. T., Sheikh, R., Hasan, S., & Rashid, K. R. (2026). Explainable sentiment analysis: A comparative study of BERT, DistilBERT, and TF-

IDF on the IMDB dataset using LIME and SHAP. *2026 IEEE 2nd International Conference on Quantum Photonics, Artificial Intelligence & Networking (QPAIN)*, 1–6. https://doi.org/10.1109/QPAIN69676.2026.11546130

Krishna, R., & Menzies, T. (2018). *Bellwethers: A baseline method for transfer learning*. arXiv. https://arxiv.org/abs/1703.06218

Krishnakumar, S., & Abdou, T. (2020). Towards interpretable and maintainable supervised learning using Shapley values in arrhythmia. *Proceedings of the 30th Annual International Conference on Computer Science and Software Engineering (CASCON '20)*, 23–32. https://doi.org/10.5555/3432601.3432605

Limbola, P., Dwivedi, A., Shrivastava, N., & Shrivastava, A. K. (2026). Advanced sentiment analysis of IMDB movie reviews using NLP preprocessing, TF-IDF and classical machine learning models. *2026 International Conference on Emerging Trends in Information, Communication & Systems (ICETICS)*, 1–6. https://doi.org/10.1109/ICETICS69505.2026.11651294

Lundberg, S. (2018). *SHAP (SHapley additive exPlanations) documentation*. https://shap.readthedocs.io/en/latest/

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, 142–150. http://www.aclweb.org/anthology/P11-1015

Ruder, S. (2019). *Sentiment analysis – NLP-progress*. https://nlpprogress.com/english/sentiment_analysis.html.

Shetty, A. M., Hegde K, R., Shabari, Kamath, S., & Vidyavathi, K. (2026). Sentiment analysis on IMDB reviews: A comparative study of bag-of-words, deep learning, and transformer models.

*2026 International Conference on Artificial Intelligence and Data Engineering (AIDE)*, 798–804. https://doi.org/10.1109/AIDE69088.2026.11544304